实验五　边缘检测与图像分割

一、实验目的与要求

1．了解图像边缘检测和分割的意义和方法，巩固所学理论知识。

2．掌握图像边缘检测方法。

3．掌握图像分割方法。

4．讨论不同算子对边缘检测效果的影响。

二、实验原理及知识点

1、边缘检测

边缘(edge)是指图像局部强度变化最显著的部分。主要存在于目标与目标、目标与背景、区域与区域(包括不同色彩)之间，是图像分割、纹理特征和形状特征等图像分析的重要基础。图像强度的显著变化可分为：

1）阶跃变化函数：即图像强度在不连续处的两边的像素灰度值有着显著的差异；

2）线条（屋顶）变化函数：即图像强度突然从一个值变化到另一个值，保持一较小行程后又回到原来的值。图像的边缘有方向和幅度两个属性,沿边缘方向像素变化平缓,垂直于边缘方向像素变化剧烈。边缘上的这种变化可以用微分算子检测出来,通常用一阶或二阶导数来检测边缘。各种算子的存在就是对这种导数分割原理进行的实例化计算， 如 SOBEL 算子， PREWITT 算子， LOG 算子，以及 Canny 算子。

下面给出边缘检测的 MATLAB 和 Python 函数:

Matlab: output = edge(input, method) 使用 method 指定的边缘检测算法检测图像 input 中的边缘。即 method 可以为'sobel'，'prewitt'，'log'，'canny'。

Python: 基于 scikit-image 图像处理包直接调用以下函数。

output = filters.sobel(input)；output = filters.prewitt(input)；output = filters.laplace(input)；output = feature.canny(input,sigma)。其中：input 是需要处理的图像，sigma 越小，canny 检测的边缘线条越细小。

2、图像分割

在图像分析中，通常需要将关心的目标从图像中提取出来，这种从图像中将某个区域与其他部分进行分离并提取出来的处理，就是图像分割。因为图像分割实际上就是区分图像中的“前景目标”和“背景”，所以通常又称之为图像的二值化处理。总的来说，图像分割是

图像识别和图像分析的基本前提步骤，图像分割的质量好坏直接影响后续图像处理的效果。

本实验采取的分割方法为阈值分割。阈值法的基本思想是基于图像的灰度特征来计算一个或多个灰度阈值，并将图像中每个像素的灰度值与阈值相比较，最后将像素根据比较结果分到合适的类别中。因此，该类方法最为关键的一步就是按照**某个准则函数**来求解最佳灰度阈值 T。

下面给出二值化阈值分割的 MATLAB 和 Python 函数:

Matlab: output = im2bw(input, level)。其中 input 为待分割图像，level 为灰度阈值。

Python: output = cv2.threshold (input, thresh)，其中 thresh 为灰度阈值。

3、实验完整代码

Matlab 边缘检测完整代码：

```
clear;close all;

%加载图片并读取图片信息
srcImage=imread('testimg\lenna.jpg');

%将图像转化为灰度图
grayImage=rgb2gray(srcImage);

%分别使用\Sobel\Prewitt\log\Canny 算子进行边缘检测
imageSobel=edge(grayImage,'sobel');
imagePrewitt=edge(grayImage,'prewitt');
imageLog=edge(grayImage,'log');
imageCanny=edge(grayImage,'canny');

%显示检测图像
subplot(2,3,1);
imshow(grayImage);
title('原图像');

subplot(2,3,2);
imshow(imageLog);
title('log');

subplot(2,3,3);
```

```
imshow(imageSobel);
title('sobel');

subplot(2,3,4);
imshow(imagePrewitt);
title('prewitt');

subplot(2,3,5);
imshow(imageCanny);
title('canny');
```

Matlab 图像分割完整代码：

```
clear;
close all;

%加载图片并显示
srcImage=imread('testimg\lenna.jpg');
subplot(1,3,1);
imshow(srcImage);
title('original');

%固定阈值 T
T = 100;
level = T/255;
bw = im2bw(srcImage, level);
%显示分割图像
subplot(1,3,2);
imshow(bw);
title('T=100');

%迭代式阈值分割
zmax=max(max(srcImage));%取出最大灰度值
zmin=min(min(srcImage));%取出最小灰度值
tk=(zmax+zmin)/2;
bcal=1;
```

```
[m,n]=size(srcImage);
while(bcal)
    %定义前景和背景数
    iforeground=0;
    ibackground=0;
    %定义前景和背景灰度总和
    foregroundsum=0;
    backgroundsum=0;
    for i=1:m
        for j=1:n
            tmp=srcImage(i,j);
            if(tmp>=tk)
                %前景灰度值
                iforeground=iforeground+1;
                foregroundsum=foregroundsum+double(tmp);
            else
                ibackground=ibackground+1;
                backgroundsum=backgroundsum+double(tmp);
            end
        end
    end
    %计算前景和背景的平均值
    z1=foregroundsum/iforeground;
    z2=foregroundsum/ibackground;
    tktmp=uint8((z1+z2)/2);
    if(tktmp==tk)
        bcal=0;
    else
        tk=tktmp;
    end
    %当阈值不再变化时,说明迭代结束
end
disp(strcat('迭代的阈值:',num2str(tk)));%在 command window 里显示出 :迭代的阈值:阈值
newI=im2bw(srcImage,double(tk)/255);%函数 im2bw 使用阈值（threshold）变换法把灰度图像（grayscale image）
```

```
subplot(1,3,3);
imshow(newI);
title('迭代分割');
```

Python 边缘检测完整代码：

```
from skimage import io,filters,img_as_ubyte, feature
import matplotlib.pyplot as plt

# 图像读取
img = io.imread('./testimg/lenna.jpg', as_gray=True)   # 灰度图
plt.subplot(2, 3, 1), plt.imshow(img),plt.title('original')

# sobel 边缘检测
edges = filters.sobel(img)
edges_sobel = img_as_ubyte(edges)   # 浮点型转成 uint8 型
plt.subplot(2, 3, 2), plt.imshow(edges_sobel), plt.title('sobel') # 显示图像
# prewitt
edges = filters.prewitt(img)
edges_prewitt = img_as_ubyte(edges)   # 浮点型转成 uint8 型
plt.subplot(2, 3, 3), plt.imshow(edges_prewitt), plt.title('prewitt') # 显示图像
# log
edges = filters.laplace(img)
edges_log = img_as_ubyte(edges)   # 浮点型转成 uint8 型
plt.subplot(2, 3, 4), plt.imshow(edges_log), plt.title('log') # 显示图像
# canny
edges = feature.canny(img, sigma=1.0)
edges_canny = img_as_ubyte(edges)   # 浮点型转成 uint8 型
plt.subplot(2, 3, 5), plt.imshow(edges_canny), plt.title('canny') # 显示图像

plt.show()
```

Python 图像分割完整代码：

```
import cv2 as cv
import numpy as np
```

```
import matplotlib.pyplot as plt

def Iterate_Thresh(img, initval, MaxIterTimes=20, thre=1):
    """ 阈值迭代算法
    Args:
        img: 灰度图像
        initval: 初始阈值
        MaxIterTimes: 最大迭代次数，默认 20
        thre：临界差值，默认为 1
    Return:
        计算出的阈值
    """
    mask1, mask2 = (img > initval), (img <= initval)
    T1 = np.sum(mask1 * img) / np.sum(mask1)
    T2 = np.sum(mask2 * img) / np.sum(mask2)
    T = (T1 + T2) / 2
    # 终止条件

    if abs(T - initval) < thre or MaxIterTimes == 0:
        return T
    return Iterate_Thresh(img, T, MaxIterTimes - 1)

img = cv.imread('./testimg/lenna.jpg', 0)
# 计算灰度平均值
initthre = np.mean(img)
# 固定阈值 100
out1 = cv.threshold(img, 100, 255, cv.THRESH_BINARY)[1]
# 迭代阈值
thresh = Iterate_Thresh(img, initthre, 50)
out2 = cv.threshold(img, thresh, 255, cv.THRESH_BINARY)[1]
# 作图
plt.subplot(1,3, 1), plt.imshow(img), plt.title('original')
plt.subplot(1,3, 2), plt.imshow(out1), plt.title('fixed')
plt.subplot(1,3, 3), plt.imshow(out2), plt.title('iter')
plt.show()
```

三、实验内容及步骤

Matlab 边缘检测部分：

1．利用 imread()函数读取一幅灰度图像；

2．利用 rgb2gray ()命令转为灰度图像；

3．利用 edge ()进行边缘检测；

4．利用 imshow()命令显示检测后的图像；

5．观察不同检测算子的结果；

8．将每一步的函数执行语句拷贝下来，写入实验报告，并且将得的图像效果保存（可以截图）。

Matlab 图像分割部分：

1．利用 imread()函数读取一幅灰度图像；

2．设置不同的 T，观察分割结果；

3．设置动态阈值，观察分割结果；

4．利用 imshow()命令显示分割后的图像；

5．观察不同阈值下分割的质量；

8．将每一步的函数执行语句拷贝下来，写入实验报告，并且将得的图像效果保存（可以截图）。

Python 边缘检测部分：

1．利用 cv2.imread()函数读取一幅灰度图像，并用 plt.imshow()函数显示；

2．利用 np.mean()计算灰度平均值；

3．利用 filters. prewitt ()函数将原始灰度图像进行边缘检测，并显示；

4．利用 filters. laplace ()函数将原始灰度图像进行边缘检测，并显示；

5．利用 feature.canny ()函数将原始灰度图像进行边缘检测，并显示；

6．观察不同检测算子的结果；

7．将每一步的函数执行语句拷贝下来，写入实验报告，并且将得的图像效果保存（可以截图）。

Python 图像分割部分：

1．利用 cv2.imread()函数读取一幅灰度图像，并用 plt.imshow()函数显示；

2．利用 np.mean()计算灰度平均值；

3．利用 cv2.threshold()函数进行分割；

4．固定阈值进行分割，并显示；

5．使用动态阈值分割并显示；

6．观察不同阈值选取下分割的好坏；

7．将每一步的函数执行语句拷贝下来，写入实验报告，并且将得的图像效果保存（可以截图）。

四、考核要点

1、 熟悉在 Matlab / Python 中如何使用不同算子对图像进行边缘检测和分割、如何显示图像，熟悉相关的处理函数。

2、 设置固定阈值 T 完成分割，并比较不同阈值的结果。参考教材中的迭代阈值分割法，设计程序并完成分割。

五、实验仪器与软件

(1) PC 计算机

(2) MatLab 软件/语言包括图像处理工具箱(Image Processing Toolbox) ;
Python 语言环境/编辑器/图像处理 scikit-image 扩展包(skimage)/ matplotlib 扩展包

(3) 实验所需要的测试图片

六、实验报告要求

描述实验的基本步骤，用数据和图片给出各个步骤中取得的实验结果和源代码，并进行必要的讨论，必须包括原始图像及其计算/处理后的图像。

七、思考题

(1) 比较四种算子的边缘检测结果，并分析及其本质原因；

(2) 分析提取算子里不同参数对结果的影响；

(3) 比较迭代阈值分割与固定阈值分割结果，并分析原因；

(4) 简述 OTSU 分割方法的原理，及其与迭代阈值分割的异同点。

八、实验图像

Fig.1 lenna.jpg